

ĐẠI HỌC BÁCH KHOA HÀ NỘI
Trường Công nghệ Thông tin và Truyền thông



BÁO CÁO PROJECT CUỐI KỲ

Đề tài 2.5: Virtual Smart Waste Management Gateway Hệ thống IoT Gateway ảo quản lý thùng rác thông minh

Môn học: IT6130 - Lập trình và Ảo hóa cho IoT

Họ và tên	MSSV
Nguyễn Cao Lương (SV1)	20251244M
Ngô Văn Quang (SV2)	20251243M
Trương Tuấn Nghĩa (SV3)	20251196M

Hà Nội, tháng 6 năm 2026

Mục lục

1	Đặt vấn đề và mục tiêu đề tài	2
2	Kiến trúc hệ thống và luồng dữ liệu	2
3	Cấu trúc project và danh sách service	3
4	Thiết kế MQTT topic và message format	4
5	Virtual Sensor: mô phỏng có xu hướng	5
6	Virtual Actuator và an toàn nhiều lớp	6
7	Edge Gateway: kiểm tra hợp lệ, chuẩn hóa và lưu trạng thái	7
8	Rule Engine: tập luật phát hiện bất thường	8
9	Ghi InfluxDB, vòng đời thu gom và phát hiện offline	9
10	REST API	11
10.1	Thuật toán dự báo thời điểm thùng đầy (OLS Linear Regression)	13
11	Tích hợp ThingsBoard Cloud: uplink và RPC	14
12	Cấu hình ngưỡng rule engine khi đang chạy	17
13	Grafana Dashboard	17
14	Triển khai bằng Docker Compose	18
15	Kiểm thử hệ thống	19
16	Khó khăn, bài học và hướng phát triển	21
17	Trả lời câu hỏi bắt buộc trong báo cáo	22
18	Kết luận	23
A	Bảng biến môi trường đầy đủ	24
B	Kịch bản nguy cơ cháy minh họa	25

1. Đặt vấn đề và mục tiêu đề tài

Mô hình thu gom rác theo lịch cố định bộc lộ hai bất cập song song. Một là lãng phí: xe vẫn chạy đến những thùng chưa đầy. Hai là rủi ro: thùng quá tải lại không được ghé qua đúng lúc, gây tràn rác và mất vệ sinh đô thị. Ngoài ra, thùng rác là không gian kín, nơi khí methane sinh ra từ rác hữu cơ phân hủy có thể tích tụ đến mức nguy hiểm mà không ai giám sát, tiềm ẩn nguy cơ cháy nổ.

Nhóm em xây dựng hệ thống Virtual IoT Gateway quản lý thùng rác thông minh nhằm giải quyết ba bất cập đó: theo dõi mức đầy, khối lượng, nồng độ khí và nhiệt độ theo thời gian thực; tự động khóa nắp, nén rác, báo động và phát tín hiệu điều xe khi cần; đồng thời đồng bộ hai chiều với ThingsBoard Cloud để vận hành tập trung từ xa. Toàn bộ thiết bị vật lý, từ cảm biến đến cơ cấu chấp hành, được giả lập bằng chương trình Python chạy trong container Docker, đúng theo yêu cầu đề bài đặt ra cho năm đề tài project cuối kỳ của môn học.

Mục tiêu cụ thể:

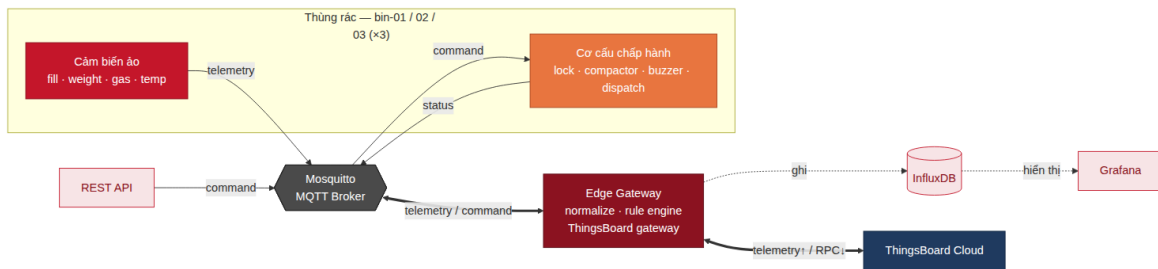
1. Lập trình nhiều virtual sensor và virtual actuator mô phỏng các thùng rác khác nhau.
2. Thiết kế topic hierarchy và message format riêng cho bài toán quản lý rác thải.
3. Xây dựng Edge Gateway chuẩn hóa dữ liệu, chạy rule engine phát hiện sự kiện bất thường.
4. Ghi telemetry, event và trạng thái thiết bị vào InfluxDB, trực quan hóa bằng Grafana.
5. Tích hợp ThingsBoard Cloud theo hai chiều: đẩy telemetry lên (uplink) và nhận lệnh điều khiển từ xa (RPC).
6. Cung cấp REST API để truy vấn trạng thái và gửi lệnh thủ công.
7. Container hóa toàn bộ hệ thống, triển khai bằng một lệnh Docker Compose duy nhất.

Phân công nhóm:

Vai trò	Họ và tên	Nhiệm vụ chính
SV1	Nguyễn Cao Lương	Virtual sensor, virtual actuator, thiết kế topic/message
SV2	Ngô Văn Quang	Edge gateway, rule engine, danh sách thu gom, ghi InfluxDB
SV3	Trương Tuấn Nghĩa	Tích hợp ThingsBoard, REST API, Docker Compose, Grafana, README

2. Kiến trúc hệ thống và luồng dữ liệu

Hệ thống mô phỏng một khu vực gồm ba thùng rác bin-01, bin-02, bin-03. Mỗi thùng có một cảm biến ảo đo mức đầy, khối lượng, nồng độ methane, nhiệt độ và trạng thái đổ nghiêng, cùng một cơ cấu chấp hành ảo điều khiển khóa nắp, bộ nén rác, còi báo động và tín hiệu điều xe thu gom. Một Edge Gateway xử lý cục bộ tại biên, đồng thời đóng vai trò cầu nối đẩy dữ liệu lên ThingsBoard Cloud.



Hình 1: Kiến trúc lại của hệ thống: cảm biến và cơ cấu chấp hành giao tiếp qua Mosquitto; Edge Gateway chuẩn hóa dữ liệu, chạy rule engine, ghi InfluxDB và đồng bộ hai chiều với ThingsBoard Cloud; Grafana đọc InfluxDB để hiển thị; REST API cho phép truy vấn và điều khiển thủ công.

Luồng dữ liệu chính:

1. Cảm biến publish telemetry dạng JSON lên waste/{bin}/sensor/telemetry mỗi 5 đến 7 giây.
2. Broker chuyển tiếp bản tin đến Edge Gateway. Gateway kiểm tra hợp lệ, chuẩn hóa rồi chạy rule engine.
3. Gateway publish dữ liệu đã chuẩn hóa, sự kiện bất thường (nếu có), lệnh điều khiển xuống actuator, và ghi cả ba loại dữ liệu này vào InfluxDB.
4. Actuator nhận lệnh, kiểm tra an toàn, cập nhật trạng thái nội bộ rồi publish lại trạng thái mới.
5. Grafana đọc InfluxDB để hiển thị dashboard; một tiến trình riêng (tb-gateway) đẩy telemetry và sự kiện lên ThingsBoard, đồng thời chuyển lệnh RPC từ cloud thành command MQTT cục bộ.

Điểm cốt lõi của kiến trúc này là tính lai: Edge Gateway vừa tự ra quyết định ngay tại biên, đảm bảo độ trễ thấp và vẫn hoạt động khi mất kết nối Internet, vừa gộp nhiều thùng rác vào một kết nối ThingsBoard Cloud duy nhất để giám sát tập trung. Mọi giao tiếp giữa các thiết bị đều đi qua broker MQTT, không thiết bị nào nối trực tiếp với thiết bị khác; broker đóng vai trò bus nội bộ tại biên, còn gateway là cầu nối duy nhất lên tầng lưu trữ và đám mây.

3. Cấu trúc project và danh sách service

```

iot-waste-management/
|-- docker-compose.yml
|-- .env.example
|-- mosquitto/config/mosquitto.conf
|-- virtual_sensor/      (sensor.py, Dockerfile, requirements.txt)
|-- virtual_actuator/    (actuator.py, Dockerfile, requirements.txt)

```

```

|-- iot_gateway/
|   |-- gateway.py           rule engine orchestrator + vong
|   lap bao tri
|   |-- rule_engine.py       tap luat (ham thuan)
|   |-- state_store.py       trang thai + debounce + danh
|   sach thu gom
|   |-- influx_writer.py      ghi InfluxDB
|   '-- tb_gateway.py        cau noi ThingsBoard
|-- gateway_api/             (api.py, FastAPI REST API)
|-- grafana/                 (provisioning datasource + dashboard)
'-- docs/                    (topic-design.md, bao cao, anh chup)

```

Bộ service khai báo trong docker-compose.yml gồm mosquitto, influxdb, waste-gateway, ba cặp sensor-bin-01/02/03 và actuator-bin-01/02/03, grafana, gateway-api, và tb-gateway (chỉ khởi động khi kích hoạt profile thingsboard).

Mỗi service tự chứa mã nguồn, requirements.txt và Dockerfile riêng để Docker Compose build độc lập. Hai container waste-gateway và tb-gateway dùng chung codebase iot_gateway/ nhưng chạy hai entypoint khác nhau, gateway.py và tb_gateway.py. Về mặt logic, cả hai cùng thuộc thành phần Edge Gateway mà đề bài mô tả; việc tách thành hai process chỉ nhằm phân chia trách nhiệm giữa xử lý cục bộ và cầu nối cloud.

4. Thiết kế MQTT topic và message format

Cây topic được thiết kế theo từng thùng, mỗi thùng có một nhóm topic riêng:

```

waste/{bin_id}/
|-- sensor/telemetry      # fill/weight/gas/temp tho tu sensor
|-- sensor/reset         # gateway bao da thu gom, sensor
                           reset
|-- actuator/command      # gateway gui lenh dieu khien
|-- actuator/status       # actuator phan hoi trang thai
|-- gateway/normalized    # du lieu da chuan hoa (gateway
                           publish)
'-- gateway/event         # su kien bat thuong (gateway
                           publish)

```

Bản tin telemetry (waste/bin-01/sensor/telemetry):

```

1 {
2   "device_id": "sensor-bin-01", "area_id": "district-1", "
      bin_id": "bin-01",
3   "fill_level": 88.0, "weight_kg": 70.4, "methane_ppm": 350.0,
4   "temperature": 33.0, "lid_status": "closed", "tilt": false,

```

```

5   "timestamp": "2026-06-10T10:00:00Z"
6 }

```

Bản tin command (waste/bin-01/actuator/command):

```

1 { "bin_id": "bin-01", "target": "dispatch", "action": "on",
2   "reason": "bin_full", "timestamp": "2026-06-10T10:00:05Z" }

```

Đặt bin_id vào cấu trúc topic, không phải trong payload, cho phép gateway dùng một lệnh subscribe wildcard waste/+sensor/telemetry để bắt telemetry của mọi thùng cùng lúc. Việc tách riêng ba nhóm sensor/, actuator/ và gateway/ giúp việc gỡ lỗi đơn giản hơn: chỉ cần subscribe một nhánh là xem được toàn bộ một loại dữ liệu. Mọi bản tin đều ở dạng JSON và luôn kèm bin_id cùng timestamp để phục vụ truy vết.

5. Virtual Sensor: mô phỏng có xu hướng

Yêu cầu của đề bài là mức đầy phải biến đổi có xu hướng theo thời gian, không random độc lập giữa các chu kỳ, vì một thùng rác thật không thể tự voi rồi đầy lại nếu không có xe đến thu gom.

```

1 def _simulate_fill(self):
2     if not self.locked:
3         noise = random.gauss(0, 0.1)                # nhiều nhỏ
4         # quanh xu hướng
5         self.fill_level += self._fill_rate + noise # fill_rate
6         # > 0 nên luôn tăng
7         self.fill_level = max(0.0, min(100.0, self.fill_level))
8     self.weight_kg = self.fill_level * KG_PER_FILL_PERCENT +
9         random.gauss(0, 0.3)

```

Nồng độ methane có tương quan thuận với mức đầy theo công thức nền base = 50 + fill_level * 5, kèm khoảng ba phần trăm xác suất mỗi chu kỳ xảy ra hiện tượng tăng đột biến từ ba trăm đến năm trăm ppm, mô phỏng túi khí bị vỡ. Nhiệt độ có thể leo thang từ hai mươi đến ba mươi lăm độ khi nồng độ methane vượt bốn trăm ppm, tạo thành đường dẫn dẫn tới sự kiện cháy. Thùng bị đánh dấu đổ nghiêng khi khối lượng vượt năm mươi lăm kilôgam. Khi nhận được lệnh {"action": "reset"} trên topic sensor/reset, sensor đặt mức đầy về khoảng 0,5 đến 3 phần trăm, mô phỏng việc vừa được thu gom.

Một chi tiết đáng chú ý được bổ sung trong quá trình hoàn thiện: sensor lắng nghe luôn topic actuator/status của chính thùng mình. Khi nắp bị khóa (lock = on, do quá tải), mức đầy và khối lượng sẽ ngừng tăng cho đến khi mở khóa hoặc thu gom xong, vì không ai có thể bỏ thêm rác vào một thùng đã khóa nắp. Nhờ vậy hành động khóa nắp có tác động thật lên mô phỏng, không chỉ là một cờ trạng thái hiển thị suông.

```
(base) ➔ iot-waste-management git:(main) ✖ docker compose logs --tail 15 sensor-bin-01
sensor-bin-01 | [sensor-bin-01] fill= 21.9% weight= 17.5kg methane= 170.4ppm temp= 29.7°C tilt=False
sensor-bin-01 | [sensor-bin-01] fill= 22.9% weight= 18.7kg methane= 165.5ppm temp= 27.4°C tilt=False
sensor-bin-01 | [sensor-bin-01] fill= 23.7% weight= 18.8kg methane= 177.1ppm temp= 31.5°C tilt=False
sensor-bin-01 | [sensor-bin-01] fill= 24.5% weight= 19.3kg methane= 198.9ppm temp= 31.3°C tilt=False
sensor-bin-01 | [sensor-bin-01] fill= 25.2% weight= 20.1kg methane= 176.1ppm temp= 29.9°C tilt=False
sensor-bin-01 | [sensor-bin-01] fill= 25.9% weight= 20.3kg methane= 162.1ppm temp= 29.9°C tilt=False
sensor-bin-01 | [sensor-bin-01] fill= 26.9% weight= 21.3kg methane= 212.8ppm temp= 30.2°C tilt=False
sensor-bin-01 | [sensor-bin-01] fill= 27.9% weight= 22.0kg methane= 213.8ppm temp= 29.5°C tilt=False
sensor-bin-01 | [sensor-bin-01] fill= 28.6% weight= 23.0kg methane= 205.3ppm temp= 28.3°C tilt=False
sensor-bin-01 | [sensor-bin-01] fill= 29.4% weight= 23.7kg methane= 670.9ppm temp= 29.7°C tilt=False
sensor-bin-01 | [sensor-bin-01] fill= 30.0% weight= 23.9kg methane= 225.4ppm temp= 31.6°C tilt=False
sensor-bin-01 | [sensor-bin-01] fill= 30.9% weight= 24.6kg methane= 197.8ppm temp= 31.2°C tilt=False
sensor-bin-01 | [sensor-bin-01] fill= 31.7% weight= 25.2kg methane= 221.2ppm temp= 30.1°C tilt=False
sensor-bin-01 | [sensor-bin-01] fill= 32.5% weight= 25.8kg methane= 205.4ppm temp= 34.5°C tilt=False
sensor-bin-01 | [sensor-bin-01] fill= 33.3% weight= 26.5kg methane= 211.7ppm temp= 28.9°C tilt=False
```

Hình 2: Log của sensor-bin-01 cho thấy mức đầy tăng dần qua từng chu kỳ, đúng yêu cầu mô phỏng có xu hướng.

Toàn bộ cấu hình của sensor, bao gồm AREA_ID, BIN_ID và PUBLISH_INTERVAL, đều đọc từ biến môi trường theo nguyên tắc mười hai yếu tố (12-factor app), không hard-code trong mã nguồn. Hằng số KG_PER_FILL_PERCENT được chọn bằng 0,8 một cách có chủ đích, đủ lớn để một thùng gần đầy vượt được cả ngưỡng quá tải (60 kg) và ngưỡng đổ nghiêng (55 kg); nếu chọn hằng số nhỏ hơn, hai tình huống này sẽ không bao giờ xảy ra và phần rule engine tương ứng không có cơ hội được kích hoạt.

6. Virtual Actuator và an toàn nhiều lớp

Actuator nhận lệnh, kiểm tra tính hợp lệ, áp dụng một lớp an toàn ngay tại thiết bị, rồi mới cập nhật trạng thái và phản hồi.

```
1 if bin_id and bin_id != BIN_ID:                                # 1. lệnh
    không phải của thùng mình
2     return
3 if target not in VALID_TARGETS:                                # 2. validate
    target
4     _publish_error(client, f"invalid_target:{target}", reason)
    ; return
5 if action not in VALID_ACTIONS:                                # 3. validate
    action
6     _publish_error(client, f"invalid_action:{action}", reason)
    ; return
7 # 4. SAFETY CHECK: cam nen rac khi dang bao chay
8 if target == "compactor" and action == "on" and state["buzzer"] == "on":
9     _publish_error(client, "safety_block:
        compactor_during_fire_risk", reason); return
10 state[target] = action                                         # 5. áp dụng +
    publish status
```

```
11 client.publish(TOPIC_STATUS, json.dumps(build_status_payload()
    ), qos=1)
```

```
(base) → iot-waste-management git:(main) ✗ curl -s -X POST http://localhost:8001/bins/bin-01/command -H "Content-Type: application/json" -d '{"target": "buzzer", "action": "on", "reason": "demo"}'
{"status": "command sent", "command": {"bin_id": "bin-01", "target": "buzzer", "action": "on", "reason": "demo", "timestamp": "2026-06-24T20:25:28Z"}}
(base) → iot-waste-management git:(main) ✗ curl -s -X POST http://localhost:8001/bins/bin-01/command -H "Content-Type: application/json" -d '{"target": "compactor", "action": "on", "reason": "demo"}'
{"status": "command sent", "command": {"bin_id": "bin-01", "target": "compactor", "action": "on", "reason": "demo", "timestamp": "2026-06-24T20:25:46Z"}}
(base) → iot-waste-management git:(main) ✗ docker compose logs --tail 20 actuator-bin-01
actuator-bin-01 | [actuator-bin-01] Khởi động – kết nối mosquitto:1883
actuator-bin-01 | [actuator-bin-01] Đang chờ lệnh trên waste/bin-01/actuator/command ...
actuator-bin-01 | [actuator-bin-01] Kết nối MQTT broker thành công
actuator-bin-01 | [actuator-bin-01] Subscribe: waste/bin-01/actuator/command
actuator-bin-01 | [actuator-bin-01] CMD nhận: {'bin_id': 'bin-01', 'target': 'buzzer', 'action': 'on', 'reason': 'demo', 'timestamp': '2026-06-24T20:25:28Z'}
actuator-bin-01 | [actuator-bin-01] CMD ✓ buzzer: off → on (reason=demo)
actuator-bin-01 | [actuator-bin-01] STATUS → waste/bin-01/actuator/status
actuator-bin-01 | [actuator-bin-01] CMD nhận: {'bin_id': 'bin-01', 'target': 'compactor', 'action': 'on', 'reason': 'demo', 'timestamp': '2026-06-24T20:25:46Z'}
actuator-bin-01 | [actuator-bin-01] SAFETY BLOCK: từ chối bật compactor khi buzzer đang on (fire_risk)
```

Hình 3: Actuator thực thi lệnh hợp lệ và từ chối lệnh nguy hiểm bằng cơ chế safety block.

Quyết định an toàn được xử lý ngay tại thiết bị, không chờ gateway phản hồi, vì trong kịch bản cháy thực tế, độ trễ mạng dù nhỏ cũng có thể gây hậu quả nghiêm trọng. Đây là một ví dụ của nguyên tắc phòng vệ theo lớp: gateway đã chủ động tắt bộ nén rác khi phát hiện nguy cơ cháy, nhưng nếu vì lý do nào đó vẫn có lệnh nén rác gửi nhầm xuống, có thể do lỗi phần mềm hoặc do lệnh thủ công gửi qua REST API, actuator vẫn tự chặn lại. Một điểm thiết kế khác là actuator luôn publish toàn bộ trạng thái của bốn thiết bị sau mỗi lệnh, không chỉ gửi phản thay đổi, để gateway và ThingsBoard luôn đồng bộ kể cả khi một bản tin nào đó bị thất lạc trên đường truyền.

7. Edge Gateway: kiểm tra hợp lệ, chuẩn hóa và lưu trạng thái

Gateway là thành phần trung tâm của hệ thống, subscribe wildcard toàn bộ thùng, làm sạch dữ liệu đầu vào, lưu trạng thái mới nhất và duy trì danh sách thùng cần thu gom.

```
1 for f in _REQUIRED_FIELDS: # du truong
    bat buoc
2     if f not in raw: return None
3 if bin_from_topic and bin_id != bin_from_topic: # bin_id khop
    topic, chong gia mao
4     return None
5 datetime.strptime(raw["timestamp"], "%Y-%m-%dT%H:%M:%SZ") #
    timestamp dung ISO-8601
6 fill = max(0.0, min(100.0, float(raw["fill_level"]))) #
    clamp + phan loai fill_status
```



```

1 def update_telemetry(self, bin_id, normalized):          # lưu
   trạng thái mới nhất từng thùng
2     with self._lock:
3         self.telemetry[bin_id] = normalized
4         self._last_seen[bin_id] = time.monotonic()
5 def mark_for_collection(self, bin_id):                  # thêm vào
   danh sách cần thu gom
6     with self._lock:
7         self._collection.setdefault(bin_id, time.monotonic())
8 def due_for_collection(self, delay):                    # thùng đã
   cho qua delay giây
9     now = time.monotonic()
10    with self._lock:
11        return [b for b, ts in self._collection.items() if now
                  - ts >= delay]

```

Khâu kiểm tra hợp lệ đóng vai trò tuyển phòng thủ đầu tiên: bản tin thiếu trường, sai định dạng timestamp, hoặc bin_id trong payload không khớp với topic đều bị loại bỏ ngay, không làm gián đoạn vòng lặp MQTT đang chạy. Lớp StateStore bọc mọi thao tác đọc và ghi trong một khóa (lock), vì callback của thư viện MQTT chạy trên thread mạng riêng, còn vòng lặp bảo trì lại chạy trên thread chính; hai luồng này truy cập đồng thời vào cùng một vùng dữ liệu nên buộc phải đồng bộ hóa. Trạng thái mới nhất của từng thùng và danh sách thùng cần thu gom được lưu tại đây để phục vụ rule engine, cơ chế debounce lệnh điều khiển và việc mô phỏng vòng đời thu gom.

8. Rule Engine: tập luật phát hiện bất thường

Rule engine áp dụng một tập luật lên bản ghi telemetry đã chuẩn hóa, trả về sự kiện phát sinh và trạng thái mong muốn của từng cơ cấu chấp hành.

Luật	Điều kiện	Sự kiện	Hành động
1	fill_level > 85 (>95: critical)	bin_full	dispatch = on
2	temperature > 60	fire_risk (critical)	buzzer = on, compactor = off
3	methane_ppm > 500	gas_alert (warning)	gợi ý thông khí
4	weight_kg > 60	overweight (warning)	lock = on
5	tilt = true	bin_tilted (info)	lên lịch bảo trì

```

1 def evaluate(t: dict, th: Thresholds) -> dict:
2     events, desired = {}, {}

```

```

3     if fill > th.fill_dispatch:
4         severity = "critical" if fill > th.fill_critical else
           "warning"
5         events["bin_full"] = {...}
6         desired["dispatch"] = "on"
7     else:
8         desired["dispatch"] = "off"           # het day, tu tat (
           level-based)
9     return {"events": events, "desired": desired}

```

```

(base) → iot-waste-management git:(main) ✗ docker compose logs --tail 40 waste-gateway | grep -E "EVENT|CMD|COL
TED"
waste-gateway | [waste-gateway] EVENT bin-03: gas_alert (warning) value=592.84 thr=500.0
waste-gateway | [waste-gateway] EVENT bin-02: gas_alert (warning) value=531.51 thr=500.0
waste-gateway | [waste-gateway] EVENT bin-01: gas_alert (warning) value=670.92 thr=500.0
waste-gateway | [waste-gateway] EVENT bin-02: gas_alert (warning) value=589.86 thr=500.0
waste-gateway | [waste-gateway] EVENT bin-01: gas_alert (warning) value=571.12 thr=500.0
waste-gateway | [waste-gateway] EVENT bin-01: gas_alert (warning) value=722.91 thr=500.0
waste-gateway | [waste-gateway] EVENT bin-01: gas_alert (warning) value=719.77 thr=500.0
waste-gateway | [waste-gateway] EVENT bin-02: gas_alert (warning) value=742.81 thr=500.0
waste-gateway | [waste-gateway] EVENT bin-02: gas_alert (warning) value=755.39 thr=500.0

```

Hình 4: Gateway ghi nhận sự kiện `bin_full`, gửi lệnh `dispatch=on`, và sau đó publish lệnh reset khi xe mô phỏng đã đến thu gom.

Rule engine được thiết kế như một hàm thuần: cùng một đầu vào luôn cho ra cùng một đầu ra, không publish lên MQTT, không ghi cơ sở dữ liệu và không tự giữ trạng thái. Nhờ đặc điểm này, rule engine kiểm thử được hoàn toàn độc lập, không cần khởi động broker hay database thật. Thiết kế cũng tách bạch hai khái niệm dễ nhầm lẫn: trường `desired` mang tính theo mức, gateway dựa vào đó để debounce, chỉ gửi lệnh điều khiển khi trạng thái mong muốn thực sự thay đổi, tránh lặp lại mỗi chu kỳ; còn trường `events` mang tính cạnh, chỉ phát sinh đúng một lần tại thời điểm điều kiện chuyển từ sai sang đúng, giúp biểu đồ số lượng sự kiện theo thời gian trên Grafana phản ánh đúng ngữ nghĩa. Toàn bộ ngưỡng được đọc từ biến môi trường nên có thể chỉnh từ xa qua REST API hoặc ThingsBoard mà không cần sửa mã nguồn.

9. Ghi InfluxDB, vòng đời thu gom và phát hiện offline

Ba loại dữ liệu được ghi vào InfluxDB: `bin_telemetry` với tag `area_id`, `bin_id` và field `fill_level`, `weight_kg`, `methane_ppm`, `temperature`; `gateway_events` với tag `bin_id`, `event_type`, `severity` và field `value`, `threshold`; `actuator_status` với tag `bin_id` và bốn field tương ứng bốn thiết bị, được mã hóa về dạng số 0 và 1 để Grafana vẽ được.

```

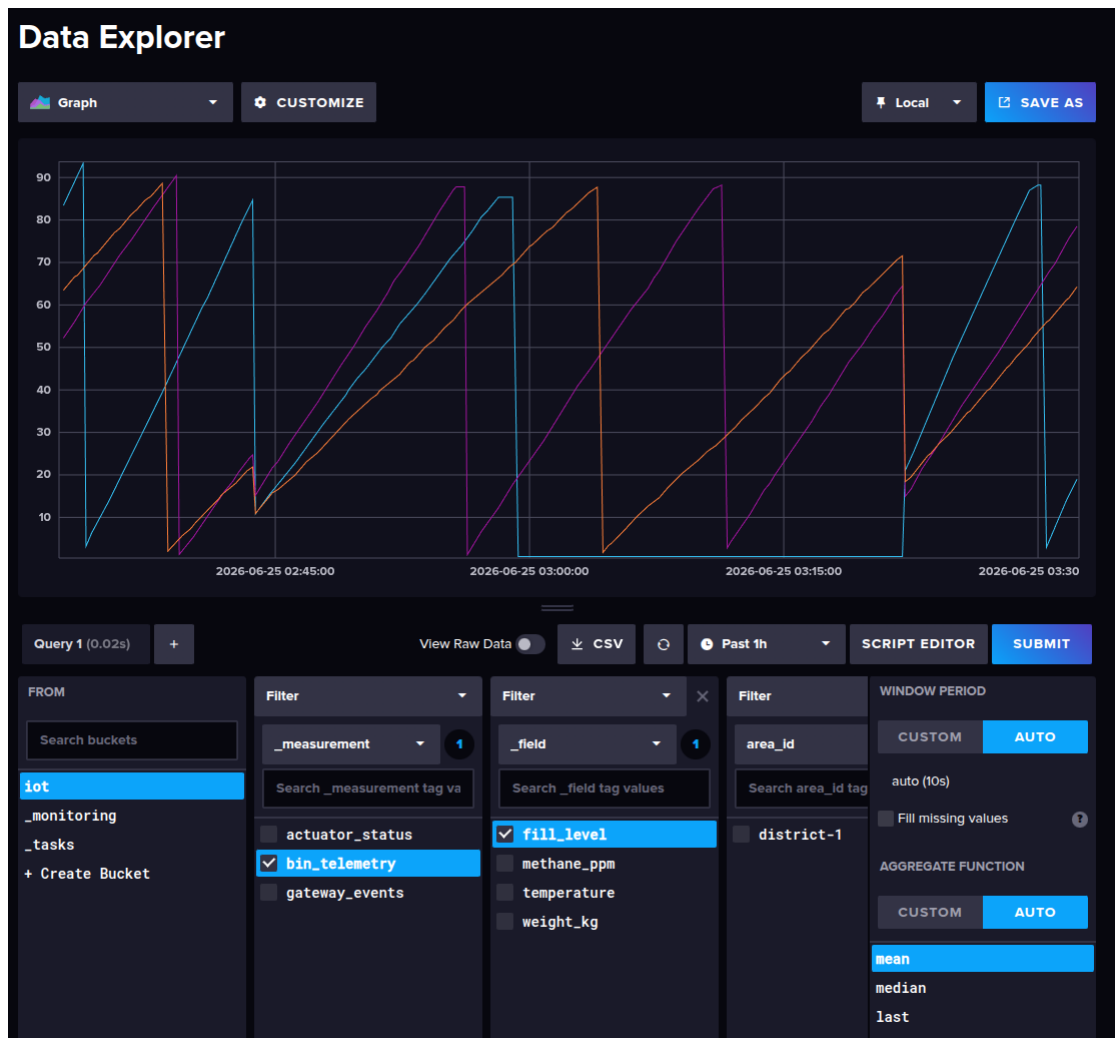
1 for bin_id in store.due_for_collection(COLLECTION_DELAY):           #
    mo phong xe toi noi
2     client.publish(f"waste/{bin_id}/sensor/reset", ...)             #
    sensor reset

```

```

3     store.clear_collection(bin_id)
4     newly_offline, recovered = store.offline_transitions(
        SENSOR_OFFLINE_TIMEOUT)
5     for bin_id in newly_offline:                                     #
        qua_han_khong_co_telemetry
6     client.publish(f"waste/{bin_id}/gateway/event",
        sensor_offline_payload)

```



Hình 5: InfluxDB Data Explorer hiển thị measurement bin_telemetry với trường fill_level theo thời gian.

Một nguyên tắc thiết kế quan trọng là lỗi ghi InfluxDB không được phép làm gateway dừng hoạt động. Việc khởi tạo client InfluxDB được bọc trong khối thử bắt lỗi; nếu cơ sở dữ liệu tạm thời không sẵn sàng, gateway vẫn tiếp tục chạy ở chế độ không ghi, rule engine và việc điều khiển actuator không bị ảnh hưởng. Vòng đời thu gom khép kín theo trình tự: thùng đầy dẫn đến lệnh `dispatch=on`, thùng được đưa vào danh sách cần thu gom, sau một khoảng thời gian mô phỏng xe đến nơi, gateway publish lệnh `reset`, sensor đặt lại mức đầy gần 0 để bắt đầu một chu kỳ mới.

10. REST API

REST API đóng vai trò một lớp giao diện HTTP cho phía biên, cho phép truy vấn trạng thái và gửi lệnh điều khiển mà không cần nói trực tiếp giao thức MQTT.

GET	/health	kiểm tra service + danh sách bin
GET	/bins	telemetry mọi nhất mọi thùng (doc InfluxDB)
GET	/bins/{id}/state	telemetry + trạng thái actuator
GET	/bins/{id}/events	sự kiện gần đây
GET	/collection/route	danh sách thùng cần thu gom ngay bây giờ
POST	/bins/{id}/command	gửi lệnh thu gom
GET/POST	/config	xem / điều chỉnh ngưỡng rule engine khi đang chạy

Waste Management Gateway API 1.0.0 OAS 3.1

/openapi.json

default

GET	/health	Health	▼
GET	/bins	List Bins	▼
GET	/bins/{bin_id}/state	Get Bin State	▼
GET	/bins/{bin_id}/events	Get Bin Events	▼
POST	/bins/{bin_id}/command	Send Command	▼
GET	/config	Get Config	▼
POST	/config	Update Config	▼

Schemas

CommandRequest > Expand all object

HTTPValidationError > Expand all object

ThresholdConfig > Expand all object

ValidationError > Expand all object

Hình 6: Trang Swagger UI tại <http://localhost:8001/docs> liệt kê và cho phép thử trực tiếp các endpoint.

Các endpoint đọc trạng thái và sự kiện truy vấn trực tiếp vào InfluxDB bằng Flux, còn các endpoint điều khiển và cấu hình publish lệnh lên broker MQTT, không gọi trực tiếp vào

gateway hay actuator. Dữ liệu đầu vào được kiểm tra bằng Pydantic, trả về mã lỗi 404 hoặc 400 theo đúng quy ước REST.

Đề bài liệt kê endpoint GET /collection/route làm danh sách thùng cần thu gom ngay tại thời điểm gọi. Hàm collection_route() tương ứng đã có sẵn trong state_store.py, nhưng vì gateway-api chạy trong một container riêng, không chia sẻ vùng nhớ với waste-gateway, nên không thể gọi trực tiếp hàm đó. Endpoint được hoàn thiện bằng cách suy ra danh sách từ giá trị fill_level mới nhất của từng thùng trên InfluxDB, đúng cách mà panel Grafana "Thùng cần thu gom hiện tại" đã làm, sắp xếp giảm dần theo mức đầy để gợi ý thứ tự ưu tiên thu gom. Cách làm này tránh phải thêm một cơ chế chia sẻ trạng thái mới giữa hai container chỉ để phục vụ một endpoint.

Một lỗi khác chỉ lộ ra trong lúc tập demo: endpoint GET /bins/{id}/state có lúc báo actuator là no_data dù thiết bị vẫn đang ở một trạng thái hợp lệ. Nguyên nhân nằm ở cửa sổ thời gian của câu truy vấn Flux. Hàm lấy telemetry mới nhất dùng range(start: -5m), hợp lý vì sensor publish liên tục mỗi năm đến bảy giây nên luôn có dữ liệu trong năm phút gần nhất. Hàm lấy trạng thái actuator lại copy nguyên cửa sổ đó, nhưng actuator chỉ publish khi nhận lệnh mới, không publish định kỳ. Một thùng đã khóa nắp từ mười lăm phút trước, nếu không có lệnh nào khác xảy ra trong khoảng đó, sẽ bị API báo nhầm thành không có dữ liệu dù trạng thái khóa vẫn còn nguyên. Khi rà lại, nhóm em thấy panel Grafana "Trạng thái thiết bị theo thùng" đã dùng đúng cửa sổ hai mươi bốn giờ cho measurement actuator_status, nên chỉ cần sửa REST API theo đúng cách đó là khớp lại. Đây là một minh chứng nữa cho việc dữ liệu theo chu kỳ và dữ liệu theo sự kiện cần chiến lược truy vấn khác nhau, dù cùng nằm trong một cơ sở dữ liệu.

Ngoài các endpoint trong đề bài, nhóm em bổ sung thêm ba endpoint vượt yêu cầu để hệ thống hoàn thiện hơn về mặt vận hành:

- GET /summary: Tổng quan tức thì của toàn hệ thống — số thùng online, offline, số cần thu gom, số ở mức nguy cấp, và trạng thái rút gọn từng bin. Phù hợp cho màn hình giám sát trung tâm hoặc webhook định kỳ.
- GET /bins/offline: Sensor offline detection theo yêu cầu nâng cao. So sánh timestamp bản ghi mới nhất với biến môi trường SENSOR_OFFLINE_TIMEOUT, trả ra hai danh sách rõ ràng: thùng đang online và thùng đã im lặng quá lâu, kèm thời gian bản ghi cuối cùng nhận được.
- GET /bins/{id}/eta: Dự báo thời điểm thùng đầy theo yêu cầu nâng cao. Lấy chuỗi fill_level trong mười lăm phút gần nhất, áp hồi quy tuyến tính OLS để tính tốc độ tăng trung bình theo phút, rồi suy ra thời điểm đến mức một trăm phần trăm. Kết quả gồm eta_minutes, eta_timestamp (ISO 8601 UTC), fill_rate_per_minute, và mức độ tin cậy (high nếu có từ năm điểm trở lên, low nếu ít hơn). Endpoint trả null khi fill_level đang giảm hoặc đứng yên, không đưa ra dự báo vô nghĩa. Thuật toán được tách thành module riêng gateway_api/eta_predictor.py để kiểm thử độc lập không cần InfluxDB.

10.1. Thuật toán dự báo thời điểm thùng đầy (OLS Linear Regression)

Bài toán đặt ra là: cho n bản đo (t_i, f_i) trong mười lăm phút gần nhất, với t_i là Unix timestamp tính theo giây và f_i là `fill_level` tính theo phần trăm, hãy ước tính thời điểm f đạt 100 %.

Bước 1 — Chuẩn hóa thời gian. Đặt $x_i = t_i - t_0$ với $t_0 = t_1$ là timestamp của điểm đo đầu tiên. Phép dịch này giữ nguyên hình học của dữ liệu nhưng tránh tràn số khi bình phương (Unix timestamp $\approx 1,7 \times 10^9$ giây, bình phương trực tiếp cho $\approx 2,9 \times 10^{18}$, gần giới hạn của `float64`).

Bước 2 — Hệ số góc OLS. Tìm đường thẳng $f = ax + b$ tối thiểu hóa tổng bình phương sai số:

$$a = \frac{SS_{xy}}{SS_{xx}}, \quad SS_{xy} = \sum_{i=1}^n (x_i - \bar{x})(f_i - \bar{f}), \quad SS_{xx} = \sum_{i=1}^n (x_i - \bar{x})^2$$

Đây là ước lượng *Best Linear Unbiased Estimator* (BLUE) theo định lý Gauss–Markov khi sai số đo không tương quan. Độ phức tạp tính toán là $O(n)$ thời gian và $O(n)$ bộ nhớ, không cần thư viện ngoài.

Bước 3 — Phòng vệ chiều. Nếu $a \leq 0$, thùng đang ổn định hoặc vừa được thu gom nên mức đầy giảm, không có ETA có nghĩa. Endpoint trả `eta_minutes: null` kèm ghi chú rõ lý do, thay vì trả một con số âm gây hiểu nhầm.

Bước 4 — Tính ETA. Giải phương trình $f_{\text{last}} + a \cdot \Delta t = 100$:

$$\Delta t = \frac{100 - f_{\text{last}}}{a} \quad (\text{giây})$$

Thời điểm dự báo $= t_{\text{last}} + \Delta t$, tính từ timestamp *điểm đo cuối cùng* (không phải thời điểm gọi API) vì f_{last} là giá trị tại điểm đó.

Ví dụ số minh họa. Giả sử năm điểm đo cách nhau 60 s, mức đầy tăng đều từ 46 % đến 50 %:

x_i (s)	f_i (%)
0	46,0
60	47,0
120	48,2
180	49,1
240	50,0

$$\bar{x} = 120, \quad \bar{f} = 48,06$$

$$SS_{xy} = (-120)(-2,06) + (-60)(-1,06) + 0 + (60)(1,04) + (120)(1,94) = 606,0$$

$$SS_{xx} = 120^2 + 60^2 + 0 + 60^2 + 120^2 = 36\,000$$

$$a = \frac{606}{36\,000} \approx 0,01683 \text{ \%/s} \approx 1,01 \text{ \%/phút}$$

$$\Delta t = \frac{100 - 50,0}{0,01683} \approx 2973 \text{ s} \approx 49,5 \text{ phút}$$

Trong điều kiện mô phỏng thực tế, sensor publish mỗi năm đến bảy giây, nên cửa sổ mười lăm phút cung cấp khoảng 128 đến 180 điểm đo. Với $n \geq 5$ điểm, endpoint trả confidence: "high"; dưới năm điểm (mới khởi động) trả "low".

Thuật toán OLS được tách thành hàm thuần `predict_eta()` trong module `gateway_api/eta_predictor.py`. Hàm không có I/O, nhận vào danh sách tuple và trả về `ETAResult`, nên có thể kiểm thử hoàn toàn độc lập không cần Docker hay InfluxDB. Bộ test `test_eta_predictor.py` gồm hai mươi test case bao phủ: dữ liệu hoàn toàn tuyến tính (kiểm tra slope chính xác đến sáu chữ số thập phân), dữ liệu nhiễu, slope âm, chuỗi phẳng, chuỗi một điểm, chuỗi rỗng, thùng đã đầy, và tính bất biến của chuẩn hóa thời gian.

11. Tích hợp ThingsBoard Cloud: uplink và RPC

Tiến trình `tb_gateway.py` đóng vai trò cầu nối, sử dụng hai kết nối MQTT song song: một nối vào Mosquitto cục bộ, một nối lên `thingsboard.cloud` bằng ThingsBoard Gateway MQTT API.

```

1  # UPLINK: doc du lieu chuan hoa tu broker, day len ThingsBoard
2  local_client.subscribe("waste/+/gateway/normalized")
3  tb_client.publish("v1/gateway/connect", json.dumps({"device":
    bin_id}))
4  tb_client.publish("v1/gateway/telemetry", json.dumps({"bin_id":
    [{"ts": ts, "values": values}]}))
5
6  # DOWNLINK: RPC tu ThingsBoard, chuyen thanh command MQTT cuc
    bo
7  def rpc_to_command(bin_id, method, params):          # setLock/
    setCompactor/setBuzzer/setDispatch
8      return {"bin_id": bin_id, "target": RPC_METHOD_MAP[method
    ],
9              "action": rpc_action_from_params(params), "reason"
    : "thingsboard_rpc"}
10 local_client.publish(f"waste/{bin_id}/actuator/command", json.
    dumps(command))

```

bin-01

Device details

?

✕

<

Details

Attributes

Latest telemetry

Calculated fields

Alarm rules

...

>

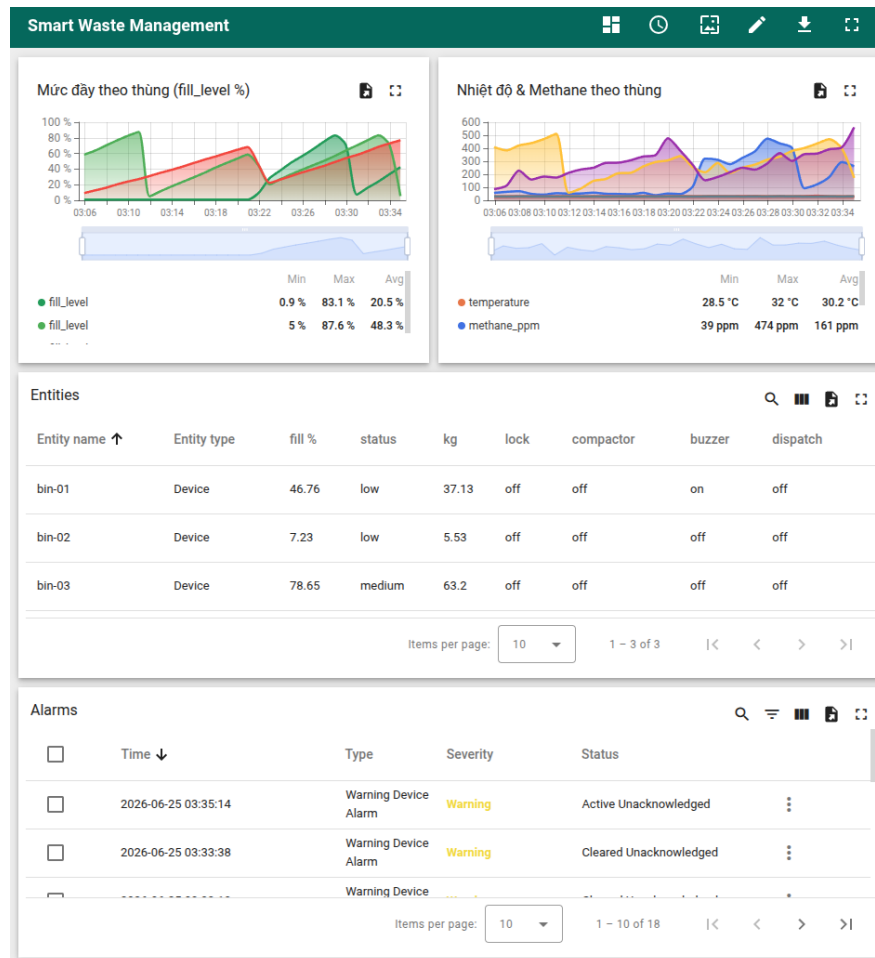
Telemetry

+

🔍

☐	Last update time	Key ↑	Value	
☐	2026-06-25 03:33:58	alarm_action	ventilation_suggested	🗑
☐	2026-06-25 03:33:58	alarm_severity	warning	🗑
☐	2026-06-25 03:33:58	alarm_threshold	500.0	🗑
☐	2026-06-25 03:33:58	alarm_type	gas_alert	🗑
☐	2026-06-25 03:33:58	alarm_value	553.45	🗑
☐	2026-06-25 03:30:18	buzzer	on	🗑
☐	2026-06-25 03:30:18	compactor	off	🗑
☐	2026-06-25 03:30:18	dispatch	off	🗑
☐	2026-06-25 03:34:38	fill_level	40.26	🗑
☐	2026-06-25 03:34:38	fill_status	low	🗑

Hình 7: Ba thiết bị con bin-01, bin-02, bin-03 tự xuất hiện dưới gateway trên ThingsBoard Cloud; tab Latest telemetry cập nhật fill_level, methane_ppm và các trường khác theo thời gian thực.



Hình 8: Rule Chain trên ThingsBoard sinh Alarm mức Critical và Warning; dashboard có nút RPC điều khiển trực tiếp từ xa.

Bridge chỉ đẩy lên ThingsBoard dữ liệu đã được chuẩn hóa và sự kiện đã qua xử lý của gateway, không đẩy telemetry thô. Vai trò ThingsBoard gateway được tách thành một container riêng tên `tb-gateway`, dùng chung codebase với `waste-gateway` nhưng chạy độc lập; về bản chất, đây chính là phần ThingsBoard gateway của Edge Gateway mà đề bài yêu cầu.

Một sự cố đáng ghi lại trong quá trình tích hợp: phiên bản đầu gửi yêu cầu `shared attributes` lên topic `v1/gateway/attributes/request` với payload `{"sharedKeys": ...}`, là định dạng dành cho API của chính thiết bị gateway, không phải API xin thuộc tính của một sub-device. ThingsBoard nhận bản tin sai định dạng và ngắt kết nối ngay sau mỗi lần kết nối thành công, tạo ra một vòng lặp connect rồi disconnect lặp lại mỗi một đến hai giây. Lỗi này không bị unit test với mock phát hiện, chỉ lộ ra khi chạy thật trên Docker, vì bản chất của nó nằm ở giao tiếp thật với dịch vụ ThingsBoard Cloud. Sau khi đổi sang đúng topic `v1/devices/me/attributes/request/1`, kết nối ổn định trở lại.

12. Cấu hình ngưỡng rule engine khi đang chạy

Mục tiêu của phần này là cho phép đổi ngưỡng rule engine từ REST API hoặc từ ThingsBoard Shared Attributes mà không cần khởi động lại container.

```

REST POST /config      +-
                        +--> topic waste/gateway/config (
                            retained)
ThingsBoard Shared Attr. +-      -> gateway.
    handle_config_update()

                                -> cập nhật
                                THRESHOLDS ngay
                                tức thì
  
```

```

1 def handle_config_update(payload):
2     for env_key, value in payload.get("thresholds", {}).items
      ():
3         field = _THRESHOLD_ENV_TO_FIELD.get(env_key)
4         if field:
5             setattr(THRESHOLDS, field, float(value)) # có
                  hiệu lực từ telemetry kế tiếp
  
```

Cả hai nguồn cấu hình cùng dùng một topic waste/gateway/config. Đối tượng THRESHOLDS là một dataclass có thể thay đổi giá trị (mutable), được dùng chung với hàm evaluate(), nên lệnh setattr có hiệu lực ngay từ bản telemetry kế tiếp. Bản tin sai định dạng hoặc giá trị không ép được về số thực sẽ bị bỏ qua một cách an toàn, không làm rule engine bị lỗi.

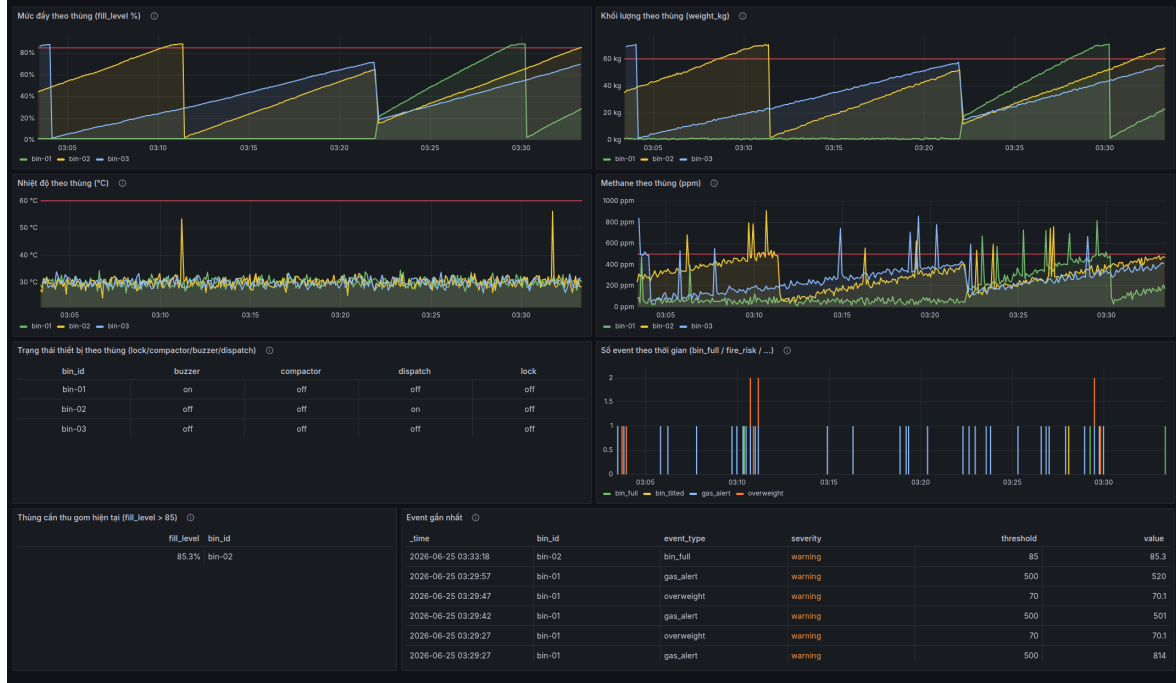
Tính năng này từng tồn tại trên giấy nhưng không có tác dụng thật: cả hai nguồn đều publish lên đúng topic, song ban đầu không thành phần nào subscribe topic đó. Sau khi bổ sung phần subscribe và xử lý ở gateway, một bộ kiểm thử đơn vị đã chứng minh việc hạ ngưỡng nhiệt độ cháy qua đúng đường dữ liệu này làm cùng một bản ghi telemetry chuyển từ trạng thái an toàn sang trạng thái fire_risk ngay lập tức.

13. Grafana Dashboard

InfluxDB được khai báo làm data source cho Grafana thông qua cấu hình provisioning tự động, gọi vào InfluxDB bằng địa chỉ http://influxdb:8086, dùng tên service thay vì localhost, với ngôn ngữ truy vấn Flux.

Dashboard gồm tám panel, đủ cả sáu nhóm mà đề bài yêu cầu: mức đầy theo thời gian cho từng thùng kèm đường ngưỡng thu gom 85 phần trăm; khối lượng theo thời gian cho từng thùng; nhiệt độ và nồng độ methane theo thùng, mỗi đại lượng một panel riêng kèm

đường ngưỡng cảnh báo; trạng thái bốn thiết bị chấp hành theo từng thùng dưới dạng bảng; số lượng sự kiện theo thời gian tách theo loại sự kiện; bảng các thùng đang cần thu gom ngay tại thời điểm xem, suy ra từ mức đầy mới nhất vượt 85 phần trăm; và bảng sự kiện gần nhất kèm mức độ nghiêm trọng.



Hình 9: Dashboard Grafana hiển thị mức đầy, nhiệt độ và methane, trạng thái thiết bị, cùng số lượng sự kiện theo thời gian thực của ba thùng rác.

Dashboard thể hiện rõ mối liên hệ nhân quả trong hệ thống: khi mức đầy của một thùng vượt ngưỡng, sự kiện `bin_full` xuất hiện ngay trên panel sự kiện và tín hiệu `dispatch` của thùng đó chuyển sang trạng thái bật. Toàn bộ chuỗi xử lý từ thiết bị, qua MQTT, qua gateway, vào InfluxDB, đến Grafana đều thông suốt và quan sát được trực tiếp trên một màn hình duy nhất.

14. Triển khai bằng Docker Compose

```
1 cp .env.example .env
2 docker compose up -d --build
3 docker compose --profile thingsboard up -d --build tb-gateway
4 docker compose ps
```

```
(base) → iot-waste-management git:(main) x docker compose ps
```

NAME	STATUS	IMAGE	PORTS	COMMAND	SERVICE	CREATED
actuator-bin-01	Up 3 hours	iot-waste-management-actuator-bin-01		"python -u actuator...."	actuator-bin-01	3 hours ago
actuator-bin-02	Up 3 hours	iot-waste-management-actuator-bin-02		"python -u actuator...."	actuator-bin-02	3 hours ago
actuator-bin-03	Up 3 hours	iot-waste-management-actuator-bin-03		"python -u actuator...."	actuator-bin-03	3 hours ago
gateway-api	Up 3 hours	iot-waste-management-gateway-api	0.0.0.0:8001->8000/tcp, [::]:8001->8000/tcp	"uvicorn api:app --h..."	gateway-api	3 hours ago
grafana	Up 2 hours	grafana/grafana:10.4.3		"/run.sh"	grafana	29 hours ago
influxdb	Up 4 hours (healthy)	influxdb:2.7	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp	"/entrypoint.sh infl..."	influxdb	29 hours ago
mosquitto	Up 4 hours (healthy)	iot-waste-management-mosquitto	0.0.0.0:8086->8086/tcp, [::]:8086->8086/tcp	"/docker-entrypoint...."	mosquitto	29 hours ago
sensor-bin-01	Up 34 minutes	iot-waste-management-sensor-bin-01	1883/tcp	"python -u sensor.py"	sensor-bin-01	35 minutes ago
sensor-bin-02	Up 34 minutes	iot-waste-management-sensor-bin-02		"python -u sensor.py"	sensor-bin-02	35 minutes ago
sensor-bin-03	Up 34 minutes	iot-waste-management-sensor-bin-03		"python -u sensor.py"	sensor-bin-03	35 minutes ago
tb-gateway	Up About an hour	iot-waste-management-tb-gateway		"python -u tb_gateway..."	tb-gateway	About an hour ago
waste-gateway	Up 34 minutes	iot-waste-management-waste-gateway		"python -u gateway.py"	waste-gateway	35 minutes ago

Hình 10: Kết quả docker compose ps: mosquitto, influxdb, waste-gateway, ba sensor, ba actuator, grafana và gateway-api đều ở trạng thái Up.

Một lệnh duy nhất dựng toàn bộ hệ thống. Mọi tham số cấu hình đọc từ biến môi trường và file .env, theo đúng nguyên tắc mười hai yếu tố (xem Phụ lục A để biết đầy đủ ý nghĩa từng biến). Các service gọi nhau bằng tên service trong mạng `iot_net` do Docker Compose tạo ra, tuyệt đối không dùng `localhost`. Tất cả mười hai service đều có healthcheck: Mosquitto và InfluxDB dùng lệnh kiểm tra tích hợp của chính chúng; Grafana dùng `wget /api/health` (image Alpine có `wget`, không có `curl`); gateway-api dùng `python3 -c urllib.request.urlopen('/health')` để gọi đúng endpoint FastAPI đã có sẵn; các service không có HTTP port (waste-gateway, sensor, actuator, tb-gateway) dùng `grep /proc/1/cmdline` để xác nhận tiến trình Python chính còn sống. Mọi service đều đặt chính sách restart: `unless-stopped`; dữ liệu InfluxDB và Grafana được giữ lại qua named volume kể cả khi container bị xóa và tạo lại. Thành phần ThingsBoard được tách theo profile riêng để chỉ khởi động khi đã có token hợp lệ, tránh lỗi không cần thiết khi chưa cấu hình ThingsBoard.

Khi cần dừng hệ thống, hai lệnh sau được dùng tùy mục đích:

```
1 docker compose down          # dừng container, giữ lại volume
   du lieu
2 docker compose down -v      # xóa cả volume (mất du lieu
   InfluxDB/Grafana)
```

15. Kiểm thử hệ thống

```
1 docker exec mosquitto mosquitto_sub -t "waste/+/sensor/
   telemetry" -v
```

```
2 docker exec mosquitto mosquitto_sub -t "waste/+ /gateway/event"
-v
```

```
(base) → iot-waste-management git:(main) ✕ docker exec mosquitto mosquitto_sub -h localhost -u iot_user -P iot_password -t "waste/+ /sensor/telemetry" -v
waste/bin-01/sensor/telemetry {"device_id": "sensor-bin-01", "area_id": "district-1", "bin_id": "bin-01", "fill_level": 79.74, "weight_kg": 63.74, "methane_ppm": 428.81, "temperature": 29.48, "lid_status": "closed", "tilt": false, "timestamp": "2026-06-24T20:28:32Z"}
waste/bin-02/sensor/telemetry {"device_id": "sensor-bin-02", "area_id": "district-1", "bin_id": "bin-02", "fill_level": 55.22, "weight_kg": 44.15, "methane_ppm": 353.53, "temperature": 27.05, "lid_status": "closed", "tilt": false, "timestamp": "2026-06-24T20:28:33Z"}
waste/bin-03/sensor/telemetry {"device_id": "sensor-bin-03", "area_id": "district-1", "bin_id": "bin-03", "fill_level": 47.64, "weight_kg": 38.63, "methane_ppm": 281.82, "temperature": 31.27, "lid_status": "open", "tilt": false, "timestamp": "2026-06-24T20:28:34Z"}
waste/bin-01/sensor/telemetry {"device_id": "sensor-bin-01", "area_id": "district-1", "bin_id": "bin-01", "fill_level": 80.5, "weight_kg": 64.05, "methane_ppm": 429.2, "temperature": 29.48, "lid_status": "closed", "tilt": false, "timestamp": "2026-06-24T20:28:37Z"}
waste/bin-02/sensor/telemetry {"device_id": "sensor-bin-02", "area_id": "district-1", "bin_id": "bin-02", "fill_level": 55.7, "weight_kg": 44.67, "methane_ppm": 316.54, "temperature": 32.41, "lid_status": "closed", "tilt": false, "timestamp": "2026-06-24T20:28:38Z"}
```

Hình 11: Subscriber nhận đúng bản tin JSON telemetry từ cả ba thùng rác.

Song song với kiểm thử thủ công bằng mosquitto_sub, hệ thống có một bộ unit test chạy bằng unittest của Python, không cần Docker, broker hay database thật:

```
1 python -m unittest discover -s iot_gateway -p "test_*.py" -v
2 python -m unittest discover -s gateway_api -p "test_*.py" -v
3 python -m unittest discover -s virtual_sensor -p "test_*.py" -v
```

Phạm vi kiểm thử bao gồm rule engine và state store với các trường hợp debounce lệnh, phát hiện cạnh sự kiện và vòng đời thu gom; cầu nối ThingsBoard với việc chuyển đổi RPC, đóng gói payload telemetry và alarm, xử lý shared attributes; module dự báo eta_predictor với hai mươi test case bao phủ đầy đủ các nhánh OLS (slope dương, âm, bằng không, chuỗi rỗng, thùng đầy, kiểm tra bất biến chuẩn hóa timestamp); REST API với việc kiểm tra đầu vào, mã lỗi 404 và 400, gửi lệnh và đọc InfluxDB ở dạng giả lập; và hành vi khóa nắp của virtual sensor. Tổng số chín mươi hai test, toàn bộ đều đạt.

```
(base) → iot-waste-management git:(main) ✕ python3 -m unittest discover -s iot_gateway -p "test_*.py" -v 2>&1 | tee test_results.txt
test_empty_method_returns_none (test_tb_gateway.TestRpcToCommand.test_empty_method_returns_none) ... ok
test_known_method_maps_to_target (test_tb_gateway.TestRpcToCommand.test_known_method_maps_to_target) ... ok
test_unknown_method_returns_none (test_tb_gateway.TestRpcToCommand.test_unknown_method_returns_none) ... ok
test_failed_connect_does_not_subscribe_or_publish (test_tb_gateway.TestTbOnConnect.test_failed_connect_does_not_subscribe_or_publish) ... ok
test_request_payload_uses_shared_keys_format (test_tb_gateway.TestTbOnConnect.test_request_payload_uses_shared_keys_format) ... ok
test_requests_attributes_via_device_level_topic_not_gateway_level (test_tb_gateway.TestTbOnConnect.test_requests_attributes_via_device_level_topic_not_gateway_level) ... ok
test_subscribes_to_response_and_push_topics (test_tb_gateway.TestTbOnConnect.test_subscribes_to_response_and_push_topics) ... ok

-----
Ran 40 tests in 0.002s

OK
[waste-gateway] CONFIG cập nhật ngưỡng runtime: {'temp_fire': 50.0, 'weight_lock': 40.0}
[waste-gateway] Bỏ qua threshold không hợp lệ TEMP_FIRE_THRESHOLD='rất nóng'
[waste-gateway] CONFIG cập nhật ngưỡng runtime: {'temp_fire': 50.0}
[tb-gateway] TB connection failed, rc=5
[tb-gateway] Connected to ThingsBoard Cloud
[tb-gateway] Connected to ThingsBoard Cloud
[tb-gateway] Connected to ThingsBoard Cloud
```

Hình 12: Kết quả chạy unit test cho rule engine, tb_gateway và REST API, tất cả đều PASS.

Triết lý kiểm thử xuyên suốt là tách phần logic thuần ra khỏi phần vào ra (I/O): các hàm tính toán trong rule engine và các hàm chuyển đổi dữ liệu trong tb_gateway đều không gọi mạng, nên kiểm thử được độc lập, nhanh và không phụ thuộc môi trường. Công cụ mosquitto_sub vẫn giữ vai trò là cách nhanh nhất để xác nhận luồng MQTT đang chạy đúng, trước khi đi sâu kiểm tra InfluxDB hay Grafana.

16. Khó khăn, bài học và hướng phát triển

- **Kết nối ThingsBoard chậm chèn.** Như đã trình bày ở Mục 11, việc gửi sai định dạng API yêu cầu shared attributes khiến ThingsBoard ngắt kết nối liên tục. Bài học rút ra là phải nắm chính xác đặc tả của từng API, không suy đoán theo cảm tính dù hai API trông tương tự nhau.
- **Lỗi chỉ lộ ra khi chạy thật.** Bộ unit test dùng mock không phát hiện được lỗi trên, vì nó nằm ở tầng giao tiếp thật với dịch vụ bên ngoài. Điều này cho thấy kiểm thử đơn vị và kiểm thử tích hợp bổ sung cho nhau, không thể thay thế nhau hoàn toàn.
- **Tính năng tồn tại trên giấy.** Topic waste/gateway/config từng được publish đầy đủ nhưng không có thành phần nào subscribe, khiến tính năng chỉnh ngưỡng runtime không có tác dụng thật dù từng dòng code liên quan đều đúng. Bài học là kiểm tra đầu cuối toàn bộ luồng dữ liệu quan trọng hơn việc kiểm tra từng module riêng lẻ.
- **Dừng tiến trình an toàn.** Lệnh docker stop gửi tín hiệu SIGTERM, không phải SIGINT mà chương trình ban đầu chỉ bắt qua KeyboardInterrupt. Nhóm đã bổ sung xử lý tín hiệu SIGTERM để đảm bảo kết nối MQTT lên ThingsBoard được đóng sạch trước khi container dừng, tránh để lại một phiên kết nối "ma" gây nhiễu cho lần khởi động kế tiếp.
- **Hằng số mô phỏng chưa phù hợp.** Hằng số KG_PER_FILL_PERCENT ban đầu được chọn quá nhỏ, khiến hai sự kiện overweight và bin_tilted không bao giờ xảy ra trong thực tế mô phỏng. Sau khi tăng giá trị này lên 0,8, rule engine kích hoạt được đúng như thiết kế.
- **Nhầm cửa sổ truy vấn giữa dữ liệu định kỳ và dữ liệu theo sự kiện.** Phát hiện trong buổi tập demo, đã trình bày chi tiết ở Mục 9: copy nguyên cửa sổ -5m từ hàm lấy telemetry sang hàm lấy trạng thái actuator khiến API báo nhầm trạng thái hợp lệ thành không có dữ liệu. Bài học là không nên sao chép tham số truy vấn giữa hai loại dữ liệu có nhịp publish khác nhau mà không xem lại từng trường hợp.

Hướng phát triển tiếp theo gồm bảo mật kết nối MQTT bằng TLS kết hợp xác thực thiết bị chặt chẽ hơn, mở rộng thuật toán tối ưu lộ trình thu gom cho quy mô nhiều khu vực và

nhiều gateway (hiện tại chỉ sắp xếp theo mức đầy, chưa tính khoảng cách địa lý hay thời gian di chuyển), nâng cao độ chính xác của mô hình dự báo thời điểm thùng đầy bằng cách bổ sung khoảng tin cậy 95% dựa trên phần dư hồi quy và áp dụng cửa sổ trượt thích nghi thay vì cửa sổ cố định mười lăm phút, cùng việc nổi thêm kênh thông báo qua email hoặc Telegram khi Alarm trên ThingsBoard chuyển sang mức nguy cấp.

17. Trả lời câu hỏi bắt buộc trong báo cáo

1. Vì sao thu gom theo nhu cầu hiệu quả hơn lịch cố định? Edge gateway hỗ trợ thế nào?

Lịch cố định không biết thùng nào thực sự đầy nên vừa lãng phí khi đi thu những thùng còn trống, vừa rủi ro khi thùng tràn đúng vào lúc giữa hai chuyến thu gom. Thu gom theo nhu cầu chỉ điều xe khi mức đầy thực tế vượt ngưỡng, dựa trên dữ liệu thời gian thực thay vì lịch trình cố định từ trước. Edge gateway hỗ trợ bằng cách giám sát liên tục mọi thùng, chạy rule engine để phát hiện thùng đã đầy, duy trì một danh sách các thùng cần thu gom, và phát tín hiệu điều xe ngay tại biên mà không phải chờ một quyết định từ trên cloud.

2. Nhóm thiết kế topic và message format cho thùng rác thông minh thế nào?

Topic được phân cấp theo mẫu `waste/{bin_id}/{nhóm}/{loại}`, với `bin_id` nằm ngay trong cấu trúc topic để gateway có thể subscribe wildcard một lần cho mọi thùng. Ba nhóm con `sensor/`, `actuator/` và `gateway/` được tách riêng để dễ theo dõi từng loại dữ liệu. Mọi bản tin đều là JSON, luôn có `bin_id` và `timestamp`; telemetry gồm các trường mức đầy, khối lượng, nồng độ methane, nhiệt độ, trạng thái nắp và cờ đổ nghiêng, như trình bày ở Mục 4.

3. Rule engine xử lý tình huống đầy, cháy, quá tải theo những luật nào?

Năm luật đã trình bày ở Mục 7: mức đầy vượt 85 phần trăm bật tín hiệu điều xe và sinh sự kiện `bin_full`, vượt 95 phần trăm thì sự kiện này nâng lên mức critical; nhiệt độ vượt 60 độ bật còi đồng thời tắt bộ nén rác, sinh sự kiện `fire_risk` ở mức critical; nồng độ methane vượt 500 ppm sinh sự kiện `gas_alert`; khối lượng vượt 60 kilôgam khóa nắp và sinh sự kiện `overweight`; thùng bị đổ nghiêng sinh sự kiện `bin_tilted` ở mức info để đưa vào lịch bảo trì. Mỗi sự kiện đều mang một mức độ nghiêm trọng rõ ràng, từ info, warning đến critical.

4. Gateway duy trì danh sách thùng cần thu gom ra sao?

Khi rule engine xác định một thùng cần điều xe, gateway gọi hàm `mark_for_collection` để lưu thời điểm thùng được đưa vào danh sách. Vòng lặp bảo trì chạy định kỳ, gọi hàm `due_for_collection` để mô phỏng thời điểm xe

thu gom đến nơi, publish lệnh reset cho sensor, rồi gọi `clear_collection` để đưa thùng ra khỏi danh sách. Hàm `collection_route` trả về danh sách đã sắp theo thời gian chờ lâu nhất, gợi ý thứ tự nên thu gom trước.

5. Gateway đóng vai trò ThingsBoard Gateway như thế nào?

Gateway sử dụng ThingsBoard Gateway MQTT API: kết nối từng thùng như một sub-device thông qua topic `v1/gateway/connect`, đẩy telemetry của nhiều thùng cùng lúc qua `v1/gateway/telemetry`, nhận lệnh điều khiển qua `v1/gateway/rpc` rồi chuyển thành command MQTT cục bộ, và nhận Shared Attributes từ cloud để cập nhật ngưỡng rule engine. Trong cách triển khai của nhóm, vai trò này chạy ở một container riêng tên `tb-gateway`, đọc dữ liệu đã chuẩn hóa từ broker cục bộ rồi mới đẩy lên cloud, như trình bày ở Mục 10.

6. Vì sao một số quyết định, ví dụ báo cháy, nên xử lý ngay tại edge?

Vì độ trễ khứ hồi lên cloud, dù chỉ vài trăm milli giây, cũng có thể gây hậu quả nghiêm trọng trong một tình huống cháy. Quyết định an toàn như bật còi và cấm nén rác được xử lý ngay tại gateway và actuator theo nguyên tắc phòng vệ nhiều lớp, nên hệ thống vẫn hoạt động đúng kể cả khi mất kết nối Internet hoàn toàn.

7. Vì sao container không nên dùng localhost để gọi service khác?

Mỗi container có một network namespace riêng biệt, nên localhost bên trong một container chỉ trỏ về chính container đó, không trỏ tới container khác dù chạy trên cùng máy host. Khi các container cùng nằm trong một mạng Docker Compose, Docker cung cấp cơ chế phân giải tên dựa trên tên service, cho phép gọi `mosquitto` hay `influxdb` và tự động được dẫn tới địa chỉ IP nội bộ tương ứng.

8. Muốn mở rộng tối ưu lộ trình thu gom toàn thành phố trên ThingsBoard, cần thêm gì?

Cần mở rộng sang nhiều khu vực với nhiều gateway, mỗi gateway dùng một token riêng; gán tọa độ địa lý cho từng thùng để hiển thị trên bản đồ của ThingsBoard; bổ sung một dịch vụ tối ưu tuyến đường, nhận danh sách thùng cần thu gom làm đầu vào và trả về thứ tự ghé thăm tối ưu cho từng xe; cùng với một Rule Chain ở mức cao hơn để tổng hợp dữ liệu từ nhiều khu vực và điều phối cảnh báo tập trung.

18. Kết luận

Nhóm em đã hoàn thiện một hệ thống IoT ảo hóa đầu cuối cho bài toán quản lý thùng rác thông minh, từ tầng thiết bị giả lập đến tầng xử lý tại biên, lưu trữ thời gian thực và giám sát trên cả Grafana và ThingsBoard Cloud. Hệ thống vận hành tự động: phát hiện thùng đầy thì

điều xe, phát hiện nguy cơ cháy thì bật còi và khóa bộ nén rác, phát hiện quá tải thì khóa nắp, đồng thời vẫn cho phép can thiệp thủ công qua REST API hoặc lệnh RPC từ xa.

Quá trình thực hiện cũng cho thấy giá trị của việc kiểm thử ở nhiều tầng khác nhau. Nhiều lỗi tinh vi, từ sai định dạng API của ThingsBoard đến một tính năng tưởng đã hoàn chỉnh nhưng thực ra không ai lắng nghe, chỉ lộ ra khi toàn bộ hệ thống được chạy thật trên Docker, chứ không thể phát hiện chỉ bằng unit test. Đây là một bài học thực tế về sự khác biệt giữa việc một đoạn mã chạy đúng cú pháp và việc cả hệ thống hoạt động đúng như thiết kế.

A. Bảng biến môi trường đầy đủ

Toàn bộ tham số của hệ thống đọc từ file `.env`, không hard-code trong mã nguồn. Bảng dưới đây liệt kê đầy đủ các biến khai báo trong `.env.example` cùng ý nghĩa và giá trị mặc định.

Biến	Mặc định	Ý nghĩa
<i>MQTT Broker</i>		
MQTT_BROKER	mosquitto	Tên service broker, dùng để các container khác kết nối
MQTT_PORT	1883	Cổng MQTT
MQTT_USER	iot_user	Tài khoản xác thực với Mosquitto
MQTT_PASS	iot_pass_2026	Mật khẩu xác thực với Mosquitto
<i>Định danh khu vực và gateway</i>		
AREA_ID	district-1	Mã khu vực địa lý, gắn vào tag InfluxDB
GATEWAY_ID	waste-gateway	Tên định danh của edge gateway trên MQTT
<i>Chu kỳ publish của sensor</i>		
PUBLISH_INTERVAL	5	Chu kỳ publish telemetry, áp dụng cho bin-01 và bin-02 (giây)
PUBLISH_INTERVAL_BIN03	7	Chu kỳ riêng của bin-03, để ba thùng không đồng pha
<i>ThingsBoard Cloud</i>		
TB_HOST	thingsboard.cloud	Địa chỉ MQTT broker của ThingsBoard
TB_PORT	1883	Cổng MQTT của ThingsBoard
TB_GATEWAY_TOKEN	(rỗng)	Access token của Gateway Device, mỗi người tự tạo riêng
<i>InfluxDB</i>		

Biến	Mặc định	Ý nghĩa
INFLUXDB_URL	http:// influxdb: 8086	Địa chỉ HTTP của InfluxDB, dùng tên service
INFLUXDB_TOKEN	dev-token- change-me	Token đọc/ghi InfluxDB
INFLUXDB_ORG	hust	Organization trong InfluxDB
INFLUXDB_BUCKET	iot	Bucket lưu toàn bộ measurement time-series
INFLUXDB_USERNAME	admin	Tài khoản đăng nhập giao diện InfluxDB
INFLUXDB_PASSWORD	admin12345	Mật khẩu đăng nhập giao diện InfluxDB
<i>Ngưỡng rule engine</i>		
FILL_DISPATCH_ THRESHOLD	85	Mức đầy (%) bắt đầu điều xe thu gom
FILL_CRITICAL_ THRESHOLD	95	Mức đầy (%) nâng sự kiện bin_full lên critical
TEMP_FIRE_THRESHOLD	60	Nhiệt độ (°C) coi là có nguy cơ cháy
METHANE_ALERT_ THRESHOLD	500	Nồng độ methane (ppm) phát cảnh báo khí
WEIGHT_LOCK_ THRESHOLD	70	Khối lượng (kg) kích hoạt khóa nắp chống quá tải
SENSOR_OFFLINE_ TIMEOUT	30	Thời gian (giây) không nhận telemetry thì coi sensor offline
COLLECTION_DELAY	60	Thời gian (giây) mô phỏng xe đến nơi sau khi vào danh sách thu gom
<i>REST API và Grafana</i>		
BIN_IDS	bin-01,bin- 02,bin-03	Danh sách bin mà REST API lặp qua khi truy vấn
GRAFANA_USER	admin	Tài khoản đăng nhập Grafana
GRAFANA_PASSWORD	admin	Mật khẩu đăng nhập Grafana

B. Kịch bản nguy cơ cháy minh họa

Phụ lục này trình bày chi tiết một lượt xử lý đầy đủ của kịch bản nguy cơ cháy, từ lúc sensor sinh ra giá trị bất thường đến lúc actuator phản hồi, để làm rõ cách các thành phần phối hợp với nhau theo đúng nguyên tắc phòng vệ nhiều lớp đã nói ở Mục 5 và Mục 7.

1. **Sensor phát hiện bất thường.** Theo công thức mô phỏng ở Mục 4, khi methane_ppm

vượt 400 ppm, có hai phần trăm xác suất mỗi chu kỳ nhiệt độ leo thang thêm hai mươi đến ba mươi lăm độ. Một lần leo thang thực tế quan sát được: `temperature = 72.5`, `methane_ppm = 520.0`.

2. **Gateway chạy rule engine.** Giá trị 72,5 độ vượt ngưỡng `TEMP_FIRE_THRESHOLD` bằng 60, nên rule engine sinh sự kiện `fire_risk` ở mức `critical`, đồng thời đặt `desired = {"buzzer": "on", "compactor": "off"}`.
3. **Gateway gửi hai lệnh xuống actuator.** Vì cả hai giá trị mong muốn đều khác trạng thái đã gửi trước đó, cơ chế `debounce` trong `state_store.py` cho phép cả hai lệnh đi qua: `{"target": "buzzer", "action": "on"}` và `{"target": "compactor", "action": "off"}`.
4. **Actuator thực thi.** Cả hai lệnh hợp lệ, không vướng `safety check` vì `compactor` đang được tắt đi chứ không phải bật lên, nên actuator cập nhật `state["buzzer"] = "on"` và `state["compactor"] = "off"`, sau đó `publish` lại toàn bộ trạng thái.
5. **Tình huống giả định: ai đó gửi nhầm lệnh bật compactor.** Có thể do thao tác tay qua REST API hoặc một lỗi phần mềm khác, giả sử lệnh `{"target": "compactor", "action": "on"}` được gửi xuống đúng lúc `buzzer` đang `on`.
6. **Actuator tự chặn.** Trước khi áp dụng, actuator kiểm tra đúng điều kiện `target == "compactor" and action == "on" and state["buzzer"] == "on"`, điều kiện này đúng, nên lệnh bị từ chối và actuator `publish {"error": "safety_block:compactor_during_fire_risk"}` thay vì áp dụng.

Điểm cần nhấn mạnh trong kịch bản này là bước cuối không phụ thuộc gateway. Gateway hoàn toàn không biết có một lệnh sai vừa được gửi, vì quyết định từ chối diễn ra cục bộ ngay tại actuator. Đây chính là lý do nhóm em gọi thiết kế này là phòng vệ nhiều lớp: lớp thứ nhất là rule engine chủ động tránh ra lệnh mâu thuẫn, lớp thứ hai là actuator tự kiểm tra lại bất kể lệnh đến từ đâu.